Distilling a coding fleet into a small self-hosted model

Paul B. Isaac’s | Rust assessment design v1.1 | 15 October 2026

Status: frozen before assessment implementation at the real timestamp in docs/design-freeze.txt. No calls, checkpoint, improvement or measured cost is claimed. Original repository history is preserved; subsequent changes go in docs/deviations.md.

1. The loop and its ceiling

Versioned tasks -> configurable teacher fleet -> isolated execution and grading -> eligible, deduplicated training examples -> adapter checkpoint -> held-out gate -> local canary -> new tasks and verified failures. Selection records which output won and why; only passing, rights-cleared training-source outputs enter supervised fine-tuning (SFT). Serving logs replenish the development queue, never the sealed assessment set.

The student can exceed an individual teacher’s pass@1 on a narrow distribution by learning complementary verified solutions, reducing avoidable errors and specialising. It may also outperform a finite fleet sample on unseen tasks through generalisation and its existing pretraining; this is a hypothesis, not a guaranteed outcome. The observed best-of-three candidate set is a reference point, not a universal theoretical upper bound. Distillation alone supplies no verified corrective answer for an item on which every candidate fails. Verifiers, new tasks or search add signal. There is no credible claim of generally surpassing frontier capability from this small round; the commercial aim is cheaper verified success at comparable quality.

2. Task supply and clean separation

Start with 96 short Rust function tasks from 24 locally authored private specification/source packs: 12 packs/48 tasks train, four/16 development, eight/32 sealed test. Assign complete packs before provider calls. A shared generator, repository ancestor or specification template is one source; changing inputs or seeds never creates a new source. Commit task, oracle, author, creation time and lineage hashes. Sources share AI authorship; source independence and algorithmic novelty remain limitations. Teachers receive specifications and public examples, never hidden tests or oracle code. Test outputs are evaluation-only.

For a week, ingest rights-cleared issue fixes from consenting private repositories and independently authored contracts, then add oracle-backed boundary cases and mutations within existing splits. Track source diversity, not only sample count. New source packs, human verification of ambiguous contracts, and budget determine replenishment. Public benchmarks remain diagnostic: new random inputs do not erase exposure to a public problem. Private provenance reduces contamination risk; it cannot prove that no model has seen equivalent algorithms.

3. Grading, cheapest first

Reject empty, truncated or unparsable responses; enforce the function interface and output schema. Next check forbidden operations and obvious harness tampering. Static checks can miss aliases and reject legitimate code: they are a fast filter, never the security boundary. Run public examples, then hidden edge/property tests and differential checks against an independently reviewed oracle. Public-example hardcoding is caught by fresh private inputs; correlated oracle bugs need a second implementation or manual review. Unresolved specifications are quarantined.

A trusted Rust controller owns expected answers, case count and verdicts outside the candidate container. Candidate stdout, exit status, test-like JSON or ’all passed’ text cannot grant success. Every required response must arrive with the right schema; timeout, crash, early exit or missing output is failure. Hidden inputs necessarily reach the candidate, but hidden test code and answers do not. Host-owned syscall traces record denied network/file probes; any prohibited attempt scores zero and is quarantined, even if caught by the program. Bound inputs, output, CPU, memory and trace volume. The extra property runs and tracing cost CPU; skip expensive layers after a definitive failure. A hash of every grader version accompanies each score.

4. Interaction and training

Baseline: one independent completion from each of three provider businesses, then choose a fully passing candidate, breaking ties by measured cost and output length. Config carries endpoint, model/revision, rate limits, token ceilings, prices and output-training permissions. Rejection sampling SFT learns the selected solution; selection reasons and rejected runs remain auditable. No passing candidate means no positive example. Save preference pairs for later; do not add DPO or online RL with this small, noisy dataset.

An optional critique experiment runs only on development sources: compare three independent calls with proposal -> critique -> revision under identical per-task dollar ceilings, including critic context, reasoning tokens, retries and grading. Run both arms on matched tasks in randomised order; compare paired verified success and cost, with intervals. Its possible benefit is repairing complementary errors; its risks are correlated mistakes, prompt injection and extra latency. Keep best-of-three unless the priced experiment justifies changing it.

Adapt Apache-2.0 Qwen2.5-Coder-0.5B-Instruct [1] with native Rust Candle LoRA, rank 8, assistant-token-only loss, short sequences and a development-selected fixed schedule. Use CPU float32 initially; the observed RTX 2080 has 8 GiB and GPU use requires a real smoke test. Measure CPU time. Train a second seed-control adapter with identical data/hyperparameters. Store base revision, tokenizer/chat template, dataset hash, seed, optimiser steps, changed-weight hashes and reload/inference evidence. These criteria survive roster changes: open runnable weights, licence compatibility, task fit, memory, stable tooling and measurable quality.

Terms are an eligibility constraint, not an inference from output ownership. DeepSeek’s API terms explicitly permit training/distillation [2]. Together’s terms assign outputs but require compliance with each hosted model’s terms [3]. Mistral’s reviewed commercial/additional terms make text output a candidate, subject to selected model/account terms [4]. Anthropic and Gemini restrict competing-model development [5,6]. Enable three providers only after retaining applicable terms/licences and a dated permission decision; ambiguous material cannot enter training. Do not bypass a provider restriction through another host.

5. Proof of learning and the gate

Select all hyperparameters and the candidate on development sources. Freeze the candidate, evaluation seeds and ship margin before opening test results. Evaluate incumbent and candidate on identical inputs, decoding settings and grader versions; count every failed generation in the denominator. Primary metric: source-macro average of task success across three predeclared decoding seeds. Tests and seeds are repeated observations, not extra independent tasks. Report individual teachers and fleet best-of-three descriptively; no post-hoc ’best teacher’ significance claim.

Let each held-out source contribute one paired mean difference. Report the mean and a two-sided 95% source-level t interval, explicitly assuming approximately independent source effects and an adequate t approximation; provide a source-cluster bootstrap sensitivity interval. Eight sources are weak evidence. Ship only if both intervals’ lower bounds exceed +2 percentage points, all security checks pass, and every task/seed is accounted for. Otherwise say NO SHIP with regression, inconclusive or invalid-evaluation reason. A predeclared non-regression slice is an additional block, never an alternative route to shipping. Use a fresh sealed set or a predeclared alpha budget for later candidates.

Report 80%-power minimum detectable effect (MDE) from simulations of this exact rule, using development-estimated discordance, clustering and seed variance, with sensitivity ranges. Sanity check: with 32 independent binary task pairs, discordance q=0.30 and a two-sided 5% test, normal MDE is about 27.1 points; clustering can worsen it. This is not the gate’s final MDE. A few extra passes or a seed-only difference is not proof; a seed-only comparison can occasionally be significant by chance. Label drill fixtures separately from actual checkpoint comparisons.

6. Execution, serving and rollback

Each run gets a fresh container: non-root UID, read-only root, no network, dropped capabilities, no-new-privileges, seccomp, PID/CPU/memory limits and a host deadline. Mount only candidate code and bounded scratch; no credentials, hidden tests or Docker socket. Preflight verifies enforcement and host-owned audit traces. Shared-kernel containers cannot promise immunity to every escape.

Rust async supervision with SQLite atomically reserves maximum billable cost, including reasoning and in-flight calls. Persist responses before completion; resume finished keys without new calls. Use bounded retries/failover. Lost responses after remote acceptance remain UNKNOWN with reservations retained; reconcile through supported provider mechanisms or stop. Exactly-once remote billing is not generally guaranteed.

Serve immutable incumbent/candidate processes locally; health and inference-check both before a 5% gated canary. Log checkpoint IDs, latency, errors, verified outcomes and cost. Safety failures or predeclared error thresholds trigger atomic routing rollback while requests drain. NO SHIP candidates stay on an evaluation route. Local drills prove rollback mechanics; sparse traffic cannot prove quality.

7. Cost assumptions and accounting

Illustrative USD plan, excluding tax/labour; assumed rates, not quotes or measurements. 96 tasks x three calls, averaging 1,500 input/750 output tokens at $1/$4 per million. Hard live budget: $20; reservations use maxima, not averages.

Line	Assumption	Planned USD
Teacher generation/evaluation	288 x (1,500 x $1 + 750 x $4) / 1,000,000	1.296
Retry headroom	25% of teacher spend	0.324
Two real adapter runs	2 x 0.5 GPU-hour x $0.50/hour	0.500
Student evaluation	1 GPU-hour x $0.50/hour	0.500
Sandbox/grading	2 CPU-hours x $0.10/hour	0.200
Storage/bundling	Allocated assumption	0.100
Total	Capacity plan, not a bill	2.920

Teacher calls dominate this plan: 7,000 tasks/week implies $94.50 for 21,000 calls before retries/compute; authoring labour may dominate total effort. Replace assumptions with actual usage/prices, credits, device-hours and invoices, including failures. Export task/model/round costs and amortised serving cost per verified success. Cost per point = round cost / gain in percentage points; non-positive gain gives N/A, inconclusive gain gives only a descriptive ratio. Keep recorded spend separate from replay API spend (zero) and replay compute.

What I am not building and why

No general coding agent, Kubernetes, pretraining, online RL or web UI: these do not test this bounded loop. Bundle real recordings, weights, adapters and runtimes for credential-free offline execution. Internet bootstrap is a separate mode; never fabricate missing evidence.

Weakest part

Evaluation power/source diversity: eight held-out source clusters give fragile interval coverage and can miss regressions. A working pipeline with NO SHIP pending more sources is credible. Extra tests/seeds improve measurement, not independent evidence.

References checked 14 October 2026: [1] Qwen official model card; [2] DeepSeek API terms 4.2; [3] Together terms 2,7; [4] Mistral commercial/additional terms; [5] Anthropic D.4; [6] Gemini API Use Restrictions. Full links in accompanying sources.md.